SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

FACULTY: Dr. D. SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I **G. Naveen Kumar Reddy** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Naveen Kumar Reddy

QUESTIONS: 05 **Total Marks:25**

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.
Tasks:
a) Identify the conceptual error in the likelihood formulation.
b) Explain how this mistake affects parameter estimation.

c) Suggest the correct mathematical formulation and reasoning.
d) How would using log-likelihood fix computational issues?

Ans:

a) Conceptual error:
The likelihood function is incorrectly defined as a sum of probabilities instead of a product of probabilities. In MLE, we compute the joint probability of independent samples, which must be a product.
b) Effect on parameter estimation:
Using a sum breaks the probabilistic foundation. It:
- Does not represent joint likelihood
- Leads to incorrect optimization objective
- Produces biased or meaningless parameter estimates even with large data

c) Correct formulation:
For dataset $D = \{x_1, x_2, \ldots x_n\}$:

$$L(\theta) = \prod_{i=1}^{n} P\,(x_i \mid \theta)$$

This assumes independence and correctly models joint probability.
d) Log-likelihood advantage:

$$\log L\,(\theta) = \sum_{i=1}^{n} \log P\,(x_i \mid \theta)$$

- Converts product → sum
- Prevents numerical underflow
- Easier differentiation for optimization

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.
Tasks:
a) List possible implementation errors in the learning algorithm.
b) Analyze how incorrect learning rate or weight updates affect convergence.
c) Identify dataset-related issues that may cause this problem.
d) Propose modifications to ensure convergence.

Ans:

a) Possible implementation errors:
- Incorrect weight update rule
- Missing bias term
- Wrong activation (not step function)
- Not updating weights only on misclassification
- Data not shuffled

b) Learning rate / weight update issues:
- Too large → oscillation, no convergence
- Too small → very slow learning
- Wrong update direction → divergence

c) Dataset-related issues:
- Data not truly linearly separable
- Noise or mislabeled samples

- Feature scaling issues

d) Modifications to ensure convergence:

- Use correct update rule:

$$w = w + \eta \cdot y \cdot x$$

- Normalize features
- Choose proper learning rate
- Remove noisy data
- Use averaged perceptron (improves stability)

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.
b) Explain the role of activation functions in gradient flow.
c) Analyze how vanishing or exploding gradients affect training.
d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Ans:

a) Possible errors:

- Incorrect derivative calculation
- Chain rule applied wrongly
- Sign errors in gradients
- Wrong loss function implementation

b) Role of activation functions:

- Enable non-linearity
- Affect gradient flow
- Sigmoid/tanh → can cause vanishing gradients
- ReLU → helps maintain gradient flow

c) Vanishing / exploding gradients:

- Vanishing → gradients ≈ 0 → no learning in early layers
- Exploding → gradients very large → unstable updates

d) Fix techniques:

- Use ReLU or Leaky ReLU
- Proper weight initialization (He/Xavier)
- Batch normalization
- Gradient clipping
- Use residual connections

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.
b) Analyze the cause of fluctuations in SGD.
c) Compare the error behavior of Mini-Batch Gradient Descent.
d) Recommend the best approach and justify with error trade-offs.

Ans:

a) Why batch GD is slow:
- Uses entire dataset per update
- High computational cost
- Not scalable for large datasets

b) Cause of SGD fluctuations:
- Uses single sample → noisy gradient
- High variance updates
- Loss curve oscillates

c) Mini-batch behavior:
- Balance between GD and SGD
- Moderate speed + reduced noise
- Smoother convergence

d) Best approach:
Mini-batch Gradient Descent is recommended because:
- Faster than GD
- More stable than SGD
- Efficient for large datasets

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.
Tasks:
a) Explain how high dimensionality increases model error.
b) Identify signs of overfitting in this scenario.
c) Analyze why distance-based learning fails in high dimensions.
d) Suggest dimensionality reduction or regularization techniques to improve performance.

Ans:

a) How high dimensionality increases error:
- Data becomes sparse
- Model overfits training data
- Requires exponentially more data

b) Signs of overfitting:
- High training accuracy
- Low test accuracy
- Large generalization gap

c) Why distance-based methods fail:
- Distances become similar in high dimensions
- Nearest neighbors lose meaning
- Poor similarity measurement

d) Solutions:
- Dimensionality reduction:
    - PCA (Principal Component Analysis)
    - t-SNE
- Regularization:
    - L1 / L2
- Feature selection
- Dropout (for neural networks)

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand